

File permissions in Linux

Project description

The research team identified that the existing file and directory permissions within the `projects` directory were not aligned with the appropriate levels of access required. To strengthen system security, it was necessary to review and correct these settings. As part of this task, I conducted checks on the current permissions and updated them to reflect the proper authorization for each file and directory.

Check file and directory details

The following example shows how Linux commands were applied to check the permissions assigned to a specific directory in the file system.

```
researcher2@9589fld618fb:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Sep 21 20:16 .
drwxr-xr-x 3 researcher2 research_team 4096 Sep 21 21:07 ..
-rw--w---- 1 researcher2 research_team  46 Sep 21 20:16 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Sep 21 20:16 drafts
-rw-rw-rw- 1 researcher2 research_team  46 Sep 21 20:16 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Sep 21 20:16 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep 21 20:16 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep 21 20:16 project_t.txt
researcher2@9589fld618fb:~/projects$
```

The first line of the screenshot shows the command that was executed, while the subsequent lines present the resulting output. This command lists all contents of the `projects` directory. By using the `ls` command with the `-la` option, a detailed view of the directory contents was generated, including hidden files. The output indicates the presence of one subdirectory named `drafts`, a hidden file named `.project_x.txt`, and five additional project files. The 10-character string displayed in the first column specifies the permission settings applied to each file and directory.

Describe the permissions string

In Linux, file permissions are represented by a 10-character string that conveys both the file type and the access rights assigned to different categories of users. Each position in the string carries specific meaning:

- **First character:** Identifies the file type. A **(d)** indicates a directory, while a hyphen **(-)** denotes a regular file.
- **Characters 2–4:** Define the permissions for the **file owner (user)**. The letters **(r)**, **(w)**, and **(x)** represent read, write, and execute privileges. A hyphen **(-)** in any position means that the corresponding permission is not granted.
- **Characters 5–7:** Specify the permissions for the **group** associated with the file, using the same **(r)**, **(w)**, and **(x)** convention.
- **Characters 8–10:** Indicate the permissions for **others**, meaning all remaining system users who are neither the owner nor part of the group.

For example, the permission string **-rw-rw-r--** assigned to the file **project_r.txt** can be interpreted as follows:

- The first character **(-)** confirms it is a regular file.
- The next three characters **(rw-)** grant the file owner read and write permissions.
- The following three characters **(rw-)** provide the same read and write permissions to members of the group.
- The final three characters **(r--)** allow all other users to read the file, but not write to or execute it.

In summary, this string shows that both the owner and group can read and modify **project_r.txt**, while other users can only view it. No one is permitted to execute the file.

Change file permissions

The organization established a policy that prohibits write access for *others* on any files. To ensure compliance, I reviewed the previously listed file permissions and identified that **project_k.txt** required modification to remove write privileges for *others*.

The screenshot illustrates the process I followed using Linux commands:

```

researcher2@9589f1d618fb:~/projects$ chmod o-w project_k.txt
researcher2@9589f1d618fb:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Sep 21 20:16 .
drwxr-xr-x 3 researcher2 research_team 4096 Sep 21 21:07 ..
-rw--w---- 1 researcher2 research_team  46 Sep 21 20:16 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Sep 21 20:16 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Sep 21 20:16 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Sep 21 20:16 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep 21 20:16 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep 21 20:16 project_t.txt
researcher2@9589f1d618fb:~/projects$ █

```

The first two lines show the commands entered, while the subsequent lines display the output. I utilized the `chmod` command, which is used to adjust file and directory permissions. The first argument specifies the permissions to be modified, and the second argument designates the target file or directory. In this case, I removed the write permission for *others* on `project_k.txt`. Following the update, I executed the `ls -la` command to verify the changes and confirm that the permissions were correctly applied.

Change file permissions on a hidden file

The research team decided to archive `project_x.txt` and required its permissions to reflect a stricter access policy. Specifically, no users should be able to modify the file, while both the owner and the group should retain the ability to read it.

The screenshot below shows how I applied Linux commands to enforce these requirements.

```

researcher2@9589f1d618fb:~/projects$ chmod u-w,g-w,g+r .project_x.txt
researcher2@9589f1d618fb:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Sep 21 20:16 .
drwxr-xr-x 3 researcher2 research_team 4096 Sep 21 21:07 ..
-r--r----- 1 researcher2 research_team  46 Sep 21 20:16 .project_x.txt
drwx----- 2 researcher2 research_team 4096 Sep 21 20:16 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Sep 21 20:16 project_k.txt
-rw----- 1 researcher2 research_team  46 Sep 21 20:16 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep 21 20:16 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep 21 20:16 project_t.txt
researcher2@9589f1d618fb:~/projects$

```

The first two lines display the commands entered, and the subsequent lines show the resulting

output. Since the file name begins with a period `.project_x.txt`, I recognized it as a hidden file within the directory.

To update the permissions, I used the `chmod` command with targeted options. I removed write privileges for the owner using `u-w`, revoked write privileges for the group with `g-w`, and then explicitly granted read access to the group with `g+r`. These adjustments ensured the file could be viewed by the owner and group but not altered, aligning with the research team's archival policy.

Change directory permissions

The organization required that only the `researcher2` account be able to access the `drafts` directory and its contents. In practice, this meant that execute permissions needed to be restricted exclusively to `researcher2`, with no other users or groups retaining that level of access.

The screenshot illustrates how I applied Linux commands to enforce this policy.

```
researcher2@9589f1d618fb:~/projects$ chmod g-x drafts
researcher2@9589f1d618fb:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Sep 21 20:16 .
drwxr-xr-x 3 researcher2 research_team 4096 Sep 21 21:07 ..
-r--r----- 1 researcher2 research_team  46 Sep 21 20:16 .project_x.txt
drwx----- 2 researcher2 research_team 4096 Sep 21 20:16 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Sep 21 20:16 project_k.txt
-rw----- 1 researcher2 research_team  46 Sep 21 20:16 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep 21 20:16 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep 21 20:16 project_t.txt
researcher2@9589f1d618fb:~/projects$
```

The output lists permissions for multiple files and directories:

- **Line 1** shows the current working directory (`projects`).
- **Line 2** points to the parent directory (`home`).
- **Line 3** represents a hidden file, `.project_x.txt`.

- **Line 4** corresponds to the *drafts* directory, which now reflects the updated, restricted permissions.

In this configuration, only `researcher2` retains execute privileges. Previously, the group also had execute permissions, so I used the `chmod` command to remove them. Because `researcher2` already possessed the necessary permissions, no additional changes were required.

Summary

I adjusted several file and directory permissions within the `projects` directory to align with the organization's required access controls. To begin, I ran the `ls -la` command to review the existing permission settings, which provided the necessary context for determining what changes were needed. Based on this review, I applied the `chmod` command multiple times to update and enforce the correct authorization levels across the files and directories.